Connected application to manage teaching resources within the context of a university merger.

Bachelor thesis conducted as completion of Bachelor's degree in Computer Science

Presented by Cyril Bousmar

Haute École Léonard de Vinci
SCIENCES ET TECHNIQUES

Department of Computer Sciences – *Paul Lambin*

Academic year 2022-2023

Acknowledgments

Words cannot express my gratitude to my professor and thesis supervisor for her invaluable patience, feedback and understanding.

I also could not have undertaken this journey without my supervisor and office mates of the IT Department of UCLouvain Saint-Louis - Bruxelles, who generously provided knowledge, expertise and the best environment I could ever have wished for. I am thankful for the trust they placed in me that allowed me to produce this work during my internship to the best of my abilities.

I am also grateful to my classmates and cohort members, for their feedback sessions, and moral support.

Lastly, I would be remiss in not mentioning my family, especially my parents and brothers, as well as my worldwide friends. Their belief in me has kept my spirits and motivation high during this process.

Contents

Introduction **7**

Project overview 8

Objectives 9

1 Context **11**

1.1 Universities and their merger 11

1.1.1 Université Saint-Louis - Bruxelles 11

1.1.2 Université catholique de Louvain 12

1.1.3 The merger 12

1.1.4 At my scale 13

1.2 Technological environment 13

1.2.1 Tools for the workflow 13

1.2.2 Development environment 14

1.2.3 S.S.O. 14

1.2.4 APIs and E.S.B. 14

1.3 Project Team 15

1.4 Meetings 16

1.4.1 IT department meeting 16

1.4.2 Project meeting 16

2 Thesis calendar, and project log book and structure **17**

3 Outsystems **21**

3.1 Definition 21
3.1.1 Its structure 21
3.1.2 Compilation 22
3.1.3 Working environments 22
3.1.4 The Forge 22
3.1.5 Management services 23
3.1.6 Forum 23
3.2 Low-code 23
3.2.1 Processes 24
3.2.2 Interface 24
3.2.3 Logic 24
3.2.4 Data 25

4 Application **27**
4.1 Terms of reference and business analysis 27
4.1.1 Design 28
4.1.2 Users and actors 28
4.2 Application design and architecture 30
4.2.1 Use cases 30
4.2.2 Database 30
4.2.3 State diagrams 33
4.2.4 Authentication 34
4.2.5 Modules of the application 34
4.2.6 Modules of the environment 38
4.2.7 API of the external provider 39
4.3 General development 40
4.3.1 Roles management 40
4.3.2 Synchronization 41
4.4 My development contribution 42
4.4.1 ReproAcademic 42

4.4.2 Reprographie_CS 43
4.4.3 ReproManagement 44
4.4.4 ReproRedirection 44
4.5 In-depth features, problems, and workaround 44
4.5.1 Mocked user 45
4.5.2 Synchronization for academics 45
4.5.3 Copy to clipboard 46
4.5.4 Saving PDF files 49
4.5.5 Inconstancy of data used in Outsystems and the ESB 49
4.5.6 Adapting to Outsystems and its workflow 50

Conclusion **51**
The application 51
The internship 52
Personal opinion 53

A Figures **57**

Bibliography **67**

Acronyms **69**

THIS PAGE INTENTIONALLY LEFT BLANK

List of Figures

A-1 Example of a wire-framed screen from the Terms of reference. This screen is part of the *ReproAcademic* module and lists all the resources of a learning unit of the current academic user. 58
A-2 Latest version of the use cases. Made for Outsystems 59
A-3 Latest version of the entity diagram. 60
A-4 State diagram for an internal resource. 60
A-5 State diagram for an external resource. 61
A-6 State diagram for resource orders. 61
A-7 Example of a server action in the *Reprographie_ CS* module to expose C.R.U.D. of the database to other modules. 62
A-8 Server action in the *Reprographie_ CS* module to synchronize academic entity with all its records based on the data provided by the E.S.B.. . 63
A-9 Client action to copy to the user clipboard the link to a specific resource. 64
A-10 Client action that redirects the user to a specific module depending on its role. 65

THIS PAGE INTENTIONALLY LEFT BLANK

Introduction

Two universities merging together is neither a common nor a trivial thing. This brings up a very specific context, that involves a lot of changes for every actor closely or even distantly related to them.

In fact, for many years, the Universié Saint-Louis - Bruxelles (USLB) has used different desktops programs for a centralized management of teaching resources while the Université catholique de Louvain (UCLouvain) has had a decentralized management for them. With the merger between the two of them approaching rapidly, a new program has been required that could work in both environments.

I integrated the project team that was tasked to develop such a program and played a role in this rarely seen transition.

This thesis gathers the fruits of my work done during my internship as a partial fulfillment for my bachelor degree in Computer Science. It conveys the vision I had of the project, but even if it was a pretty global one, it still does not mean that it describes everything that happened for the project.

This document first introduces the reader to the context in which the internship and application have been done, in Chapter 1

Then, it will briefly expose the workflow of the internship and what has been done in Chapter 2

After that, an explanation of the very specific development environment I worked in is given in order (in Chapter 3) to allow the reader to fully understand the next

chapter (Chapter 4) that concentrates on my work.

Finally, a conclusion conveys my thoughts on this internship as well as an objective analysis of the work that has been done.

Project overview

The application needs to be web reactive, all-in-one. That way, this connected application will be more user-friendly with seamless updates, and it will also be easily maintainable for the IT department.

It needs to help the management of teaching resources throughout the university. As such, teachers and students are involved as "clients" as well as the Bureau des supports de cours (Office of course materials), which is the future UCLouvain department that is in charge of all the teaching resources management.

Most of the functionalities from the old and last application need to be reworked but included in this one, and new ones have to be added.

This application, as it is meant to manage the teaching resources, needs to be ready for early September 2023 and thus, its development has to be done for the end of June 2023.

The development environment was not completely defined at start. Discussions were held to determine if it would be better to use Outsystems (O.S.) or Angular with GitHub and home environments on Virtual Machine (V.M.). In the end, O.S. was selected as university directive, even if the first choice was the other one.

Next to my work, the public market has to be set for the external provider that manage the sales and printing of resources. With that, will have to be defined all the process of integration of the two systems between them.

All the process has to be integrated in the IT project process of UCLouvain, and

the management of data, as well as the authentication process need to be adapted to the merger. Which means it also heavily depends on many other side projects meant to transfer data from USLB to UCLouvain.

Objectives

Since the new application needed a fresh start with a blank page as template, my role in this process was even more important as I was the only one without a vision polluted by the last application.

That is why, I was first tasked to produce the *Terms of reference*, *Specifications* and *Analysis* before the team could move on to the development phase.

Since many people are involved in the process of making *Terms of reference*, I would have to interact with a lot of services and departments, through meetings and presentations.

Alongside, I would have to train myself in technologies that are selected for the development in the application. As already said previously, O.S. was not the choice at first and thus, I trained myself in *Angular* first.

When all of it is done and everything set up so that the team can start the development of the application, the idea is to produce the *Academic* module for the end of my internship, as a joint effort between all team members.

That means functionalities would be distributed across the team members to develop the application simultaneously.

THIS PAGE INTENTIONALLY LEFT BLANK

Chapter 1

Context

The goal of this chapter is to present and explain all the conditions in which this work was done. Which means an explanation of the two universities is provided, as well as one for the technical environment and the project team.

1.1 Universities and their merger

Since the academical year 2016-2017, the merger between the two universities has been approved, the start of the academical year 2023-2024 has been the set goal for it to happen, and UCLouvain Saint-Louis - Bruxelles is the campus born as a result.

What follows is a brief explanation of that specific context, so that the reader may understand it better, if not already aware of the situation.

1.1.1 Université Saint-Louis - Bruxelles

Founded in 1858, in the heart of Brussels, capital of the Kingdom of Belgium and the European Union, it has grown throughout the ages by diversifying the fields of studies, creating more and more faculties to house them all.

Specialized in what we call today bachelor's degrees, and more precisely in human and social sciences, the Universié Saint-Louis - Bruxelles (USLB) is the leading French university when it comes to offer them in multilingual, involving at least French, Dutch

and English, or in English exclusively.

Turned and opened to the world with many programs such as Erasmus, in collaboration with other actors, its human resources and research grew exponentially too.[1]

1.1.2 Université catholique de Louvain

Founded in 1425 in Leuven – Belgium, the Université catholique de Louvain (UCLouvain) passed through the centuries as one of the first in the world. But in 1968, it split up into two parts: a French speaking one, and a Dutch speaking one.

This first one was relocated in Louvain-la-Neuve – Belgium while the second one stayed in Leuven.

Since then, the university has had multiple campuses all around the French speaking part of the country.

Nowadays, it is the leading French speaking university in Belgium and is ranked in the top 50 of the most international universities.[2]

1.1.3 The merger

Probably one of the reasons behind the merger is that both universities highlight and share the same three objectives. [3] [4]

Therefore, it will be easier for the two institutions to find a path to share for the future. While the Université catholique de Louvain will gain a better foothold in Brussels, the Universié Saint-Louis - Bruxelles will gain a better and greater access to

[1][7]Service communication. *Une histoire ascensionnelle.* Université Saint-Louis - Bruxelles. url: `https://www.usaintlouis.be/sl/3816.html` (visited on 03/31/2023).

[2][4]Administration des relations extérieures et de la communication. *Historique.* Université catholique de Louvain. url: `https://uclouvain.be/fr/decouvrir/historique.html` (visited on 03/31/2023).

[3][6]Service communication. *Les missions et les objectifs.* Université Saint-Louis - Bruxelles. url: `https://www.usaintlouis.be/sl/1682.html` (visited on 03/31/2023).

[4][5]Administration des relations extérieures et de la communication. *Les missions et les objectifs.* Université catholique de Louvain. url: `https://uclouvain.be/fr/decouvrir/missions-vision-valeurs.html` (visited on 03/31/2023).

the international community and will benefit from the greater resources of this first one for all its researches.

1.1.4 At my scale

At every level, adaptation is needed as processes and software used by one university or the other has been chosen to be the only one remaining. Which means that their modification is in order to assure a smooth transition.

As for myself, I worked for the IT department of Universié Saint-Louis - Bruxelles that is providing all the IT support for the university and campus to become, such as user support, software management, management of all the university IT resources, web sites, data bases and data center, etc.

1.2 Technological environment

Covered in this section will be all the explanation related to the technology used for the application.

Working on an application for a university, or in fact any big company, means that there is a very specific set of rules, environment, methodologies and procedures to follow.

1.2.1 Tools for the workflow

As for keeping track of everything, it has been decided for the team to use two softwares.

Jira

Will be used for ticketing, like a Kanban, as we are working with SCRUM methodologies. However, it has been used at its bare minimum to only structure the work as it had to take as less time as possible while using it.

A document can be found in annex, it is the print of the web page that gathers all the active and done tickets during the internship. Those with the `CB` acronyms stand for *Cyril Bousmar* and are the tickets I have done.

Out of the 42 that have been done, 36 are mine.

Confluence

Is the platform on which the application documentation will be centralized but it was not a priority, and thus, will centralize everything when the development is in a further state of completion.

1.2.2 Development environment

At first, it was decided to work with JavaScript technologies such as Angular but it has been changed after a while to fit in UCLouvain directives.

This is why I trained myself on Angular at the beginning of the project but then switched and received a training in Outsystems (O.S.).

A chapter is dedicated to this technology further in (cf. Chapter 3 - Outsystems), as it will be important to explain it in more details to better understand it, since it defines all the work that has been produced and the way it has been done.

1.2.3 S.S.O.

The university is using two kinds of Single sign-on (S.S.O.) systems for all of its applications: Shibboleth Consortium (Shibboleth) and Azure Active Directory (Azure AD). In this case UCLouvain decided to use Azure AD for the entire O.S. environment. Its implementation was already done in the development system, which made the task easier.

1.2.4 APIs and E.S.B.

All the different systems and data of UCLouvain are accessible via a single bus, the Enterprise Service Bus (E.S.B.). It is accessible via REST APIs. In the very specific

development system used for this software, an Integration Service is accessible by other applications and is used to ease the call to the E.S.B., as long as there is a generated token that allows the access for the application to the E.S.B..

A website called *API store* is accessible for UCLouvain's developer (as long as the access has been granted) that can help them to use REST calls and test them.

1.3 Project Team

Project manager Maxime Vanden Abeelen is the team leader and as such, is my referent person. He overviews the project while being actively working as a developer himself.

Associate developer Samiha Jebril is a developer that is working on other features of the application.

Consulting team As O.S. is a new environment for the university, a consulting company was hired to teach the technology to the team as well as following the development to give advice and help if ever needed.

Outsystems administrators The administration team that supervised every development of software in this environment worked with the team to provide all the Integration Services, required for the application to retrieve all the data needed.

Network architects In collaboration for this project, some network architects worked with the team as it was required to provide all the access tokens needed for the application to access the E.S.B. and S.S.O..

Business analyst, software architect and developer Those are the roles I endorsed myself while working on this project. I worked on the Terms of References then on the Business Analysis, before developing the data base and many features for the application, which implies I had to provide a Software Analysis for them too.

1.4 Meetings

Two types of meetings took place recurrently and thus will be explained here.

1.4.1 IT department meeting

Held every two weeks, it concerns all employees from the IT department and is used to know on what the others are working on and to give a feedback to the head of the department.

1.4.2 Project meeting

Held in a more random way, it was always the time to prepare for what is coming next for the project and what had already been done. It was used to allocate work within the team.

Chapter 2

Thesis calendar, and project log book and structure

This chapter relates all the work that was produced during my internship in a chronological way.

Week 1 - from 02/01 to 02/03 Insertion in the working environment with the team presentation and set up of work computer. Creating the workflow for the next weeks. Learning Angular in self-learning.

Week 2 & 3 - from 02/06 to 02/19 While completing the training in Angular, many meetings were hold to create the Terms of Reference. Those were made in collaboration between the Office of course materials, the academics and the IT department. On that note, the business analysis could proceed.

Week 4 - from 02/20 to 02/26 A first draft was made for the business analysis and the general technical analysis. Presentations were made to present the first draft of the application to other university services. The use of JavaScript technologies has been reconsidered for the first time.

Week 5 - from 02/27 to 03/05 Meetings were held to rework on the draft of the application. All the documents previously referred to were modified to comply to the

new vision of the application. Presentations for the next meetings were prepared. It has definitely been decided to switch from Angular to Outsystems.

Week 6 - from 03/06 to 03/12 The analysis still going on, it has been decided that I would start familiarize myself with Outsystems while starting developing functionalities that would not change anymore. We are talking about displaying learning units in the Academic module and also displaying resources for each learning unit.

Week 7 - from 03/13 to 03/19 As the development work in Outsystems progressed, meetings to finalize the Terms of Reference for the application were held with all the involved parties. Meetings were also held with other IT teams like the network architects and Outsystems administrators to make sure everything would be ready to insert the new software in the university environment.

Week 8 - from 03/20 to 03/26 Three days were used for the training of the team in Outsystems and the two others to adjust what had already been done to good practices. The training as it stands is meant to give to the developers all the tools they need to develop in that new environment as well as giving insight on the technology used behind it.

Week 9 - from 03/27 to 04/02 As a new meeting was held with the Outsystems administrators and network architect, the training ended after two more days. This time, the focus was one the actual application and how it could be developed properly in the environment. Changes were also made for the general software analysis as it needed to be adapted to this new system.

Week 10 - from 04/03 to 04/09 To prepare development workflow, a *Confluence* page was prepared, tickets in *Jira* were created, and a team meeting was held with the purpose of attributing them to the developers. The development started on the

Academic module by first refactoring all what had been done during the training and before to use only what was really needed.

Week 11 - from 04/10 to 04/16 Development of features continued. Focus was put on those to create, read and update internal resources in the *Academic module*. There was a need of reviewing the database as changes had been decided on meetings between the Project Manager and other IT services. Tickets in Jira were created to cover more of the work that needed to be produced in the next sprints.

Week 12 - from 04/17 to 04/23 Development of new features took place. Focus was put on creating, reading and updating external resources and listing all the resources by learning units. This concluded all the possible development for the *Academic module* as the last things required were the synchronization of data with the university database made by the Project Manager and the process to store all PDF files was not decided yet.

Week 13 - from 04/24 to 04/30 Development of features for the *Management module* started and all the major features were done: listing all the resources to manage, listing all the academics and using their functions and creating the administration dashboard as well as the edit for faculties.

Week 14 - from 05/01 to 05/07 A meeting was held with the Office of course materials to review some features and see if it would be possible to incorporate in the actual development. Some were pushed for future development but most were taken into account. Terms of Reference has been changed accordingly, as well as the analysis. Development for the last features on the *Academic module* has been conducted.

Week 15 - from 05/08 to 05/14 All my work has been documented to make it as easy as possible for the Team to continue the development based on that. Last development on key features for the *Academic module* have been done.

Week 16 - from 05/15 to 05/16 The 16th of May was actually the last day of work for me and those two days allowed me to give a complete feedback on my work to the project manager, and completing some details. I helped a lot on the synchronization of data as well.

Chapter 3

Outsystems

This chapter is meant to provide the reader the very basic comprehension over Outsystems (O.S.) which is a proprietary development system.

3.1 Definition

It is a S.A.A.S. that has been chosen by UCLouvain to fasten the development of some of its software. It provides different cloud environments shared between all developers, with many tools to manage them.

Outsystems (O.S.) is a Low-code development platform which provides tools for companies to develop, deploy and manage omnichannel enterprise applications.[1]

Applications developed in O.S. can be of multiple nature: Native web, reactive web, mobile, etc.

3.1.1 Its structure

Environments can be accessed by users and can contain multiple applications.

All applications can be divided by modules and all modules inside the same environment can possibly communicate between each other.

[1][10] Outsystems. What is Outsystems ? url:`https://www.outsystems.com/` (visited on 02/27/2023).

As everything is shared between users, it uses versioning. Everything is happening in the cloud. When a feature is developed, it can be *committed* and *pushed* with one click, and different versions mergers are easily managed.

3.1.2 Compilation

While the user develops with a low-code interface, the produced code is in Extensible Markup Language (X.M.L.) and then converted in .NET for the back-end, JavaScript (JS) for the front-end, and SQL for the database.

The code is built with algorithms that are applying patterns to make the application as efficient as possible.

3.1.3 Working environments

There are three different environments used to develop a software as of now : development, test and production.

It assures that the delivered product is working as intended and enables easy future development as well as safe bug fixes. As the application was only in its development phase when I was working, the only environment I worked on was the development one.

Since it does not require to take care of Software architecture patterns such as factories, singleton, Front Controller, Bridge, and others, focus was on how to articulate those different modules together.

3.1.4 The Forge

The ecosystem uses a store where other users can publish their features and creations. It really complements the idea of a self sufficient system that can offer ready to use functionalities for all developers.

The available modules on the Outsystems Forge are like libraries for other languages. They can be reviewed by official teams and ranked by other users. That way, it can help other developers to decide whether or not using it.

3.1.5 Management services

Linked to each working environments, services are available with O.S. to manage everything around applications. And those below are the main ones used for this project.

Users to manage users and user groups, and to assign roles.

Service Center to manage web services, times and logs. Every action is automatically logged in O.S..

Lifetime to bring the application from development to production.

Service studio is the desktop application used to develop O.S. applications, whether it be mobile, web or others.

3.1.6 Forum

It is a service that is available through O.S. that brings all its users together, so that they can ask questions and interact with each other. Except for the **Documentation**[2], it is the biggest source of information for development.

3.2 Low-code

For a reactive web application, two different types of modules are useful: *Reactive App* and *Service*. When the first one could be considered as a front-end module, the second one could be considered as a back-end one.

Service Studio provides four main sections, explained below, where all the work is concentrated to develop those modules: processes, interface, logic, and data.

[2][9]Outsystems. Documentation. url: `https://outsystems.com/documentation/` (visited on 02/27/2023).

All those sections are manipulated through a lot of drag-and-drop of widgets. Depending on the section the developer is in, widgets can be buttons, inputs, etc, but also assignment actions, if statements, etc (eg. in Figure A-8).

3.2.1 Processes

It contains all routines that could be used on the application. In this application, the section is used to synchronize data of the application with the university database regularly.

3.2.2 Interface

It is basically a front-end. It manages the user interface and the logic it can have. It can not be found in modules only meant to be back-end.

That is why, it contains different **U.I. flows** that gather *Screens* and *Blocks*, two structures that are explained below.

Screen represents a page in a web application, will all its content and logic.

Block is meant to be reused in many *Screens*, throughout the entire application.

Everything in an interface section is displayed to the developer as if it is already rendered and compiled to provide the same view as the one for the end-user.

3.2.3 Logic

As it name implies, that is the core of the function of any modules. It contains actions that are equivalent to methods in other programming languages

An action can be of three types, which just define in what part of the module it can be found and act. The *Screen action* acts on screens only (eg. collects data from a form), when a *Client action* only acts within the module (eg. navigate to another screen), and a *Server action* acts in interaction with other modules (eg. fetching data).

Actions are displayed as procedural diagrams to the developer (eg. in Figure A-8).

3.2.4 Data

This section is used to define all the database of the application and serves as a representation, as the real database is hosted by Amazon cloud services and uses specific format and nomenclature.

It helps the developer to interact with the database through C.R.U.D. operations, and can also generate entity diagrams.

THIS PAGE INTENTIONALLY LEFT BLANK

Chapter 4

Application

This chapter focuses on providing the reader a clear understanding of the application and all the work produced for and on it during the internship.

4.1 Terms of reference and business analysis

The first part of the job, as this application already exists but in another form, was to comprehend, analyze and deconstruct the existing one to create a new one from the ground while taking into account what had to be used or not as patterns for the new one.

Conducted in collaboration with all the different actors impacted by the application and its usage, it has been made through multiple meetings and presentations (documents related to them can be found in the annexes), resulting in multiple versions of it that were evolving and getting better and better. It has been written in French as it needed to be easily readable by all the other actors, and not only the IT department.

The document[1] is a compilation of all the demands of the users and the answers found by the IT department for them. It explains in details what the context of the application is, how it came to live, what are its users and actors. Alone, this document can give to the reader a very good idea of what the application is all about.

[1][2]Cyril Bousmar. *Cahier des charges - Application de gestion des supports de cours.* 1.2. Université Saint-Louis - Bruxelles. Brussels, Feb. 2023.

It details how the application will work by providing wire frames, alongside functions, that were of great help to interact with the different actors and its final users. They requested to build it that way so that it would be easier for them to follow during meetings and to keep track between them. That means that, besides Specification, this document already contains some basic analysis about the application, like defining what the different users are.

Even after the first version came out and the development started, it continued to evolve and two more major modifications where made. It was possible to make them as it involved only features that would be developed later.

That is why, specification and analysis documents from the *Application design and architecture* (cf. chapter 4.2) have multiple versions too.

This chapter will only touch on a few points of the *Terms of reference* document, as to provide an overview since it is way too long, but the reader is invited to go and read it for further information about the application.

4.1.1 Design

The design created for the application, as shown on Figure A-1, was created based on the colors of USLB but as the application is not made for this university but by UCLouvain Saint-Louis - Bruxelles campus as part of UCLouvain, the color palette will change.

The idea behind this design is to clearly separate main use cases for users so that a screen usage is fairly easy and straightforward.

It was indeed mandatory to concentrate efforts not only on the content but also on how it will be displayed as the user experience for the previous application was excruciating and needed to be improved by leaps and bonds for this one.

4.1.2 Users and actors

This section describes all the users and actors that are playing a role in the application.

Below are listed all the different types of users:

- **Academic**, is a teacher of at least one learning unit and manages its resources;
- **Student** of the university, stricto sensu. It can access to read resources of its learning units;
- **Manager**, manages all resources created by an academic. Can also endorse the role of an academic;
- **Manager administrator** is a manager but gets access to edition of cover and copyright page, emails, and faculties colors. It can also launch the archiving routine;
- **Counter user** are generally people with a student-job. Holds the counter with the stocks of resources. Has to authenticate a student and then scan the resources it has to retrieve.

The following list details the external actors:

- **External provider A.P.I.** is the access point to retrieve all the information of orders made by student on the external provider website.
- **UCLouvain E.S.B.** is the access point to retrieve all the data from the database of the university.
- **UCLouvain Azure AD** is the access point to authenticate the user.

4.2 Application design and architecture

4.2.1 Use cases

The use cases have been defined directly from the *Specifications* document[2] that can be found in annex of this thesis. It clarifies all the primary functionalities and use cases of the users for this application.

The first diagram made for Angular environment, in regards of Unified Modeling Language (U.M.L.) standards, can be found in annex.

It has since been changed quite a lot to become the Figure A-2, as it needed to take into account the new environment, and new required aspects of the application.

From the specifications, five types of users have been retained: Student, Academic, Counter, Manager, and Manager Administrator.

What is important to note about this diagram is that we can clearly see that the *Manager* has access to all the functionalities of an *Academic*, the *Manager administrator* being even above and having both of them.

That is because the *Manager* has to be able to endorse the role of an *Academic* so that it can help posting resources if ever needed.

As the authentication process in not technically and directly handled by this application, it is not mentioned here in this diagram.

4.2.2 Database

It went through a lot a changes for multiple reasons even though the analysis was already completed.

First, the development environment was meant to be Javascript at the beginning, so roles were taken into account in a table. But when switching to Outsystems, it was removed from the diagram as it is managed natively without requiring any database implementation from the developer.

[2][1]Cyril Bousmar. *Analyse technique - Application de gestion des supports de cours.* 1.2. Université Saint-Louis - Bruxelles. Brussels, Mar. 2023.

Then, diagrams were made using Outsystems entities feature which is close to U.M.L. standards.

In addition, it was first said that using the table *Users* directly is a convention in the environment but it has been modified to use intermediate tables (*Academic* and *Student*) when decisions were made that *User* table could be modified and thus, impact every application that are using it. An old database diagram can be found in annex.

Hopefully, it will not be modified again as it is now completely independent in its implementation from the rest of the environment (even if tables *User* and *SAMLSamlUser* are still used, the application is not dependent from them) and its final representation can be found in full resolution in external annexes but also as Figure A-3.

But, there are also unchanged things throughout the versions such as tables naming. What is particularly interesting is the naming for tables that are creating an m to n relation between two others. They have been named after the tables they are linking and the order is defined by the process of functions in the application.

The linked table *AcademicLearningUnit* is more often called thanks to an attribute from the *Academic* table. Thus, Academic is put in front of Learning Unit.

When the naming of all entities is straightforward (if it is not the case, the reader is invited to read the *Terms of reference*, there is still one that can be obscure: the *SAML/SamlUser*.

The *SAML/SamlUser* table comes from the environment directly and contains the F.G.S. of the user that is used as a global attribute by the university throughout its entire database.

Some tables from the database contain a version field that is used to determine if the current version that the user is editing is the latest one to prevent any concurrence.

Moreover, tables that need recurrent synchronization regularly with the databases

from UCLouvain or the external provider possess a *LatestSynchronizationDate* that is used to determine if the element has already been synchronized not too long ago or not.

Synchronization is done by the back-end of the application as a background task, every now and then, to prevent the user to spend too much time waiting for it to be completed each time he connects to the application.

Tables and their content will not be synchronized at the same time and as frequently as others. An international student is more likely to change its learning units and thus, will be synchronized more often than a learning unit in itself that should not be changed throughout the academic year.

If it ever happens that the synchronization has not been done recently enough, then it will be triggered by the user's actions as a side process.

Most of the tables have a basic relation condition on them (called *Protected* in O.S.), which means that deleting a record of an entity will only delete other records if they are linked to it. It is represented by a dark gray link on Figure A-3.

But some of them are unrestricted (called *Ignored* in O.S.), so even when a record of an entity that is linked to another is deleted, the other one will not be deleted, even if it means for it not having any more links. It is represented by a light gray link on Figure A-3 (eg. between tables **Resource** and **State**).

It is used for the state tables as those are special entities that can not be modified after the project being in production.

And finally, some records of tables are linked more strongly (called *Delete* in O.S.), and when they are deleted, it will automatically delete linked records in other entities too. It is used for **Resource** and **InternalResource** for example. That is because records of those two tables are only one to one, and could be considered as a unique global record. It is represented by a red link on Figure A-3.

4.2.3 State diagrams

State diagrams of important structures in the application such as resources have been done so that it could be easier to understand their cycle of life. Those were made following U.M.L. standards. Those diagrams represent what matches the most to the reality and needs for this application.

Internal resource

As the diagram in Figure A-4 shows, an internal resource goes through different states and those changes are made either by a user action or a routine.

When an internal resource is created, it is submitted to the Office of course materials.

Then, they can treat it and send emails to put the resource on sales and notify the academics that the resource has been treated.

The resource will then be on sales, automatically, after 48 working hours, the time for the external provider to process it.

It can be removed from the sale anytime by the Office of course materials by sending an email to the external provider.

The *Archiving routine* is launched every year, when the academic year changes. At that time, all the resources, whatever the state they are in, will then be archived.

An *Archived* resource can be *Renewed* by an academic to start its cycle of life again.

Only *Archived* or *Submitted* resources can be deleted.

Proceeding this way means that after a resource has been treated by the Office of course materials, it will not be editable. It eases the changes between states and matches to the will of the academics: a published resource for the students (so *On sales*) should not be modified.

External resource

The only difference between this one in Figure A-4 and the previous section, 4.2.3 Internal resource, is the fact that there is one more state to it. It is due to the fact that the Office of course materials has to contact publishers, outside of the application, to order the resources, before starting to sale them.

The treatment process is then longer than for internal resources and that new state exists to take it into account.

Order

While this diagram is really easy, it is still useful to explain how orders are managed.

As it can be seen on Figure A-6, after a student buys a resource on the website of the external provider and presents itself to the counter, the order is retrieved at its authentication.

When the retrieval is validated, then the state changes. The state defines if the order has to be displayed in the historic or not.

4.2.4 Authentication

This process, while extremely important, is not handled by this application directly. As a matter of fact, since the application is integrated in the Outsystems (O.S.) environment, it does not need to take care of it nor the implementation of the usage of the S.S.O. of UCLouvain as it is already implemented in the environment itself.

The tables shown in Section 4.2.2 named *User* and *SAML/SamlUser* resulted of this integration. This application simply connects those tables to the *Academic* and *Student* tables if necessary, and uses the first one directly too. Not for the consistency of the database but for the functions such as displaying names after authentication.

4.2.5 Modules of the application

As already explained in the previous chapter (cf. Chapter 3 - Outsystems), Outsystems (O.S.) applications are composed of different modules. This one has been separated

into 6: ReproAcademic, Reprographie_CS, ReproManagement, ReproCounter, ReproStudent, ReproRedirection. The names have been imposed by the administration team of Outsystems (O.S.).

Outsystems (O.S.) compiled code, single-page application in REACT, can have some security problems. To prevent any of them, any method with sensible data will be server based and the client only role will then be to communicate its request to it.

The primary reason to separate the application in those modules is to take advantage of the different functions required that depend on the user and its roles, and use them to limit the user's workplace and its accesses. Thus, it ensures a bigger security on what the current user can have access to, since the module is accessible only for a type of user; any other function for other roles is not available.

Moreover, having *Reprographie_ CS* as a back-end module only, offers a lot of advantages. It is clearer and more manageable and maintainable to have all the non-client functions at the same place. It is easier to share them between modules and knows where they are coming from, even more when they are used in more than one module.

It is even more useful to keep control of what is shared. In that way, tables and database access can only be done through functions from this module. Thus, nothing except one module can interfere with the data if it is not to read them.

What follows is a more, in-depth, description and analysis of the different modules.

ReproAcademic

This module contains all the functions related to the *Academic* use cases. Users that can access screens of this module are *Academics*, *Management*, and *Management administration*.

At launch, the user accesses a list of all its learning units. If the synchronization of the user's data or its learning units is not recent enough, a new synchronization will be done on the spot. If not, data is retrieved from the application database instantly.

The user can navigate to one of its learning units and display all the resources linked to it. It can then display screens that allow it to add, edit or see in details the resource, whether it be an internal or an external.

A screen allows it to be redirected to internal or external details for a resource when clicking on the auto-generated Uniform Resource Locators (U.R.L.) for a resource.

Forms for external and internal resources are blocks as they are used in three different screens each. The same goes for the form to add an author.

The generation of the cover page is managed by this module using an external module from the Forge of Outsystems[3].

The merge of the PDF file with the cover page (and maybe the copyrights page) is also processed using this module. All this process is happening when a user chooses to add the resource after having filled in all the form.

It has been chosen for its well known reliability, and the fact that it is considered trustworthy in the OSForge.

ReproStudent

This module can be seen as a close remake of the *Academic module* (cf. section 4.2.5). It synchronizes the data in the same way and the workflow is basically the same.

Only users having the *Student* role have access to it. They have access to a list of their learning units and can click on one to see all resources linked to it.

Two other screens allow the user to see the details of resources, download PDF and navigate to the website of the external provider.

One screen allows the user to be redirected to internal or external resource details when clicking on the auto-generated Uniform Resource Locators (U.R.L.) for a resource that was provided by an Academic.

[3][8]Leonardo Fernandes. Ultimate PDF. Outsystems Forge. url: `https://www.outsystems.com/forge/component-overview/5641/ultimate-pdf` (visited on 04/12/2023).

Reprographie_CS

This module has to be the core of the application, as its name implies: Core Service (C.S.). It is only a back-end module and is never accessed directly by the user, which means that it prevents to share sensible information or actions to the user.

It is the link between the database and the other modules. It has to contain all C.R.U.D. actions, but also all synchronization actions between the database and the I.S.'s.

It allows to share actions easily between modules from a central place while having an easy maintainable and readable code.

All routines are also defined in it.

ReproManagement

This module contains all the functions related to the *Management* and *Management administrator* use cases. Users that can access screens of this module are from the*Management*, and the *Management administration* teams.

Three main flows were defined as screens: displaying all resources, displaying all academics, and the administration part. This last one is only visible and available to the *Management administration* team.

The synchronization with the external provider is only done when necessary, which means only when the user requests to get access to data that would come from it.

The edition of mails and mailing is solely done by this module as it is already integrated to Outsystems.

ReproCounter

This module has to be in contact with the external provider A.P.I.. It could operate differently depending on the decisions that will be made by the directions.

But, independently from how it retrieves order data, it operates in the same way.

It sorts user orders into two categories: *Retrievable* and *Retrieved*.

It uses a showerhead to scan resources bar codes as texts.

The only role having access to it is *Counter*. It is done this way, most of the time, job students are doing it. But it could still be useful for some people from the Office of course materials to have access to it.

It will not be developed at first, but would be great to take the IP addresses of users into account to allow access to a limited range of them only or for addresses coming from a certain location only. That way, students working for the counter could only have access to the module on site. And it could be the same by preventing users to have access to it outside specific hours.

This security measure was thought because most users will be students and the authentication of other students presenting themselves to the counter has to be done also by typing the Numéro de matricule (Student Number) (N.O.M.A.). But, it only being an auto-incremented integer, it is easy to access any student orders. To prevent any abuses, it has to be restricted as much as it can, without removing the functionality.

It has been judged better to patch the problem before instead of doing it after. Even if every action is logged and can be monitored, it is easier to prevent users having bad behavior, instead of resolving problems after it happened.

ReproRedirection

This module is related to the auto-generated U.R.L.. It allows to redirect a user to the right module depending on the role it has. The roles *Academic*, *Student*, and *Management* will be respectively redirected to modules *ReproAcademic*, *ReproStudent*, and *ReproManagement*.

Should the resource not be available for the user or if any problem arises, the user is redirected to an exception screen.

4.2.6 Modules of the environment

All the modules below are modules already integrated in the environment and will be accessed by the application *Reprographie_ CS* module (cf. section 4.2.5) to retrieve all

the required data. Those modules are Integration Service (I.S.) and will work as a broker between the application and the E.S.B.. They will only be accessed as part of synchronization processes.

Those modules are using Fichier Général du Signalétique (General Signage File) (F.G.S.), a global identifier used across the whole university for all persons registered in it.

What follow is a brief explanation of what they are. For further information, the reader is invited to read the *Specification* document[4] provided in annex.

UCLEmployee_IS gives access to data related to academics and their learning units.

UCLLearningUnit_IS gives access to data related to learning units.

UCLOrganisation_IS gives access to data related to organizations such as faculties inside the university.

UCLStudent_IS gives access to data related to students and their learning units.

4.2.7 API of the external provider

As the process to create the contract between the external provider and the university is long, the final A.P.I. that would be accessed by this application was not defined.

Because of that, it has been decided that it would be the last thing to consider and work on.

[4][1]Cyril Bousmar. *Analyse technique - Application de gestion des supports de cours.* 1.2. Université Saint-Louis - Bruxelles. Brussels, Mar. 2023.

4.3 General development

Going through phases, it has been decided to go from one module to the other one while building skills, confidence and routine on this new environment of development.

As such, the *ReproAcademic* module would be the first one to be developed and then would come *ReproStudent* as it is really similar. After that, *ReproManagement* could be worked on, to finish the development on the *ReproCounter*. *Reprographie_CS* would be developed in parallel as it regroups features for all of them.

While this plan established, it could not be completely executed as milestones for the development of modules could not be reached always in time due to the procedures in the University or the merger. Therefore, some features were developed way in advance compare to what was initially plan so that development time would not be wasted while waiting to get something from other parties.

This chapter will not describe all the work that has been done in the development phase in details as it would be far too long. That is why, only some relevant, and judged interesting features, will be explained in detail to provide the reader with an understanding of the process and the depth of thoughts and complexity of work that has been done.

4.3.1 Roles management

Roles are very important as it defined which type of user can have access to which part of the application.

The roles defined for the application are the same as the one in the section 4.2.1 - Use cases: **Counter**, **Management**, **Management administrator**, **Student**, and **Academic**.

Each screen in the application has a set of roles defined. Which means that if the user has not a role that can have access to it, it will automatically be redirected to an exception page.

Same goes for some action that can be taken.

For the *Management administration* case, if a user from the *Management* team is connected but has not the administration role, he will not be able to see buttons to access the administration page.

The management for the roles is done in the *Users* application made by Outsystems and linked to the three environments for this application. There, roles can be assigned to users and groups of users.

4.3.2 Synchronization

By design, this application needs to be independent from any other part of the systems. Which means that even, if the data cannot be retrieved from the database of the university through the E.S.B., it should still work.

That is why the synchronization is done as a routine and is not only an action triggered when the user connects itself to the application.

If by any chance, it could not have been done for too long, at the time when the user connects to the application, then the synchronization for him and related records to him in entities are manually updated.

Entities that need to be synchronized with the university database (as explained in the Chapter 4.2.2 - Database) are the following ones: **Academic**, **Student**, **LearningUnit**, **Faculty**.

Server actions exist for every entity cited above and synchronize every record one at a time to ensure the veracity of the information it contains. They have been fragmented so that they can be launched on a set of records or a specific record of an entity.

As shown in Figure A-8, the server action is calling in a loop another server action, that will check the consistency of the information of a specific record, based on the F.G.S., from the list of records that it received from the E.S.B., after using the `GetEmployees` action.

4.4 My development contribution

As I integrated the philosophy and mechanics of Outsystems way faster, had either no holidays nor had I to work on other tasks at the same time, the development I produced has been far more consequent than that from team members.

4.4.1 ReproAcademic

It has been the main focus and has been nearly finished for it to go from the development to the testing phase.

When the internship ended, the only feature that was not completely done was the upload of PDF files.

It is due to the fact that a decision still needed to be taken to decide where the files would be uploaded to. As the decision came to me on the last day of the internship, it was too late for me to work on it without taking the risk to let something not properly finished behind me.

Other than that, all other aspects of this module have been done.

And here is the list of the screens created for the module.

`LearningUnits` that lists all the learning units of the academic.

`LearningUnit` that lists all the resources of the learning unit.

`ResourceDetails` that redirects to the right resource.

`AddInternalResource`, `EditInternalResource`, `DetailsInternalResource` that displays the internal resource whether it needs to be created, edited or just seen.

`AddExternalResource`, `EditExternalResource`, `DetailsExternalResource` that displays the external resource whether it needs to be created, edited or just seen.

To fully understand the quantity of work that has been produced, the reader is invited to go and read the *Terms of reference* document [5] provided in annex. More specifically, chapter *Prototypes d'écrans - Enseignants*, and also the use-case diagram, shown in Figure A-2, with all features for *Academic*.

[5] [2]Cyril Bousmar. *Cahier des charges - Application de gestion des supports de cours.* 1.2. Université Saint-Louis - Bruxelles. Brussels, Mar. 2023.

4.4.2 Reprographie_CS

As it is this project backbone's module, I worked on it alongside the other ones, since other modules' functionalities require to communicate with the back-end and the database, such as: getting all resources in a table, adding a new author, getting the details of a resources, or updating it, etc.

The work produced here is entirely server actions, that are shared to other modules to allow them to communicate and resolve all the actions like the ones cited just above.

Here is a small example. As we do not want to expose the entire database to other modules across the entire environment, every table of the database (which is defined and accessible in this module only) is set the "expose to read only". Which means that any other C.R.U.D. action will not be accessible, except for the read one. For them to be accessible, server actions inside this very module have to be created and exposed to other modules.

As it can be seen in the Figure A-7, the server action is set "Public" which means it can be accessed by other modules. This action enables other modules to create a resource with an external resource linked to it.

Therefore, not only can the `Create` action be accessible to other modules, it can also be more controlled and define clearly what is required for this action to be taken.

It actually create to entries in the table and not just one but it can also test the sent data before doing so. As this action has automatic rollback, it ensures that the created data is handled properly right away.

Those kinds of server actions are playing the roles of middle-ware while taking into account the business part of the application at the same time.

Moreover, I worked on the completion of the synchronization of data for academics and learning units as it was part of the module I was assigned to. Which means I worked on server actions previously mentioned in Chapter 4.3.2 - Synchronization.

It was done when I had to create a procedure[6] for the `LearningUnits` screen of the *ReproAcademic* module, when the user accesses it as default page. This procedure can be found it the annex.

4.4.3 ReproManagement

For this module, the main user functions have been done: managing resources with the `AllResources` screen, searching through users and navigate to their environment with the `Academics` screen, and the dashboard for administration with the `Administration` screen.

An in-depth explanation of those screens can be found in the *Terms of reference* document[7], more specifically in chapter *Prototypes d'écrans - Reprographie*, that are linked to some features in the use-case diagram shown in Figure A-2.

4.4.4 ReproRedirection

As it is a module for which the only purpose is to redirect the user to the right module, taking its role into account, there is only one screen, `redirect` that checks the role of the user (using server actions linked to the roles and auto-generated by O.S.) that was send to this link, and that redirects it to another module screen (always called `ReproRedirection`) depending on its role.

4.5 In-depth features, problems, and workaround

This section will discuss how some features were implemented by identifying the problem and finding a workaround. Some improvements are also discussed when possible.

The first two discussions focus on the architecture of the application and its processes, when the next two ones focus on how to develop certain features in O.S..

[6][3]Cyril Bousmar. *Technical analysis of LearningUnits.* Université Saint-Louis - Bruxelles. Brussels, Mar. 2023.

[7][2]Cyril Bousmar. *Cahier des charges - Application de gestion des supports de cours.* 1.2. Université Saint-Louis - Bruxelles. Brussels, Mar. 2023.

The last two discuss the difficulties to integrate this new environment in the context of the merger.

4.5.1 Mocked user

This is a very wide addressed problem in many systems that arouse in this case because the development environment contains developer users only but all the developed features are meant to be used by others (eg. academics for the *ReproAcademic* module).

The resolution was here really simple.

`Site properties` variables exist in O.S. and are accessible in the code while they can be managed and modified in the **Service center**.

And all the functionalities developed require the F.G.S. of the user as a proof of its identity.

Therefore, two site properties were set:

- `AcademicIsMocked` is `true` if the mock is required.
- `AcademicMockedFGS` is a randomly picked F.G.S. of an academic user.

That means, whenever the `AcademicIsMocked` is true, the current user will use the `AcademicMockedFGS` instead of its own.

This will always be the case when the application is in the development environment. When the application is in production, then the `AcademicIsMocked` will be set to `false` and the F.G.S. used will be the one of the current user.

4.5.2 Synchronization for academics

When an academic user is authenticated, the data available to him need to be accurate and in its latest version. To make it happen, as already discussed in sections 4.3.2 and 4.4.2, a routine is created and works in the background without the user noticing it.

But what if the user is connected a bit before the routine happens and that the data unfortunately changes ?

To make sure it does not happen, the synchronization can be launched for this specific user only. And this process starts by checking how old the `LastSynchronizationDate` is compared to the actual date and time. This process starts when the user reaches the screen `LearningUnits` which is the default screen for the *ReproAcademic* module.

The acceptable difference between the two of them is a criteria that can be changed depending on which entity the check is done. In this case, it has been set to 24 hours.

But since this process can be a bit long, two measures were taken.

Firstly, a parameter in front-end, `IsSynchronized`, can be set to `true` after the first check so that, if the user comes back to the screen, no further check is required for the time of its session.

Secondly, a message is displayed to the user when the process of synchronization starts, so that he can understand why he has to wait a bit. As the user can either be an academic or a manager (that is "impersonating" an academic), the message varies to respectively:

- *Welcome XX ! We are synchronizing the learning units for you.*
- *Using the account of XX. Synchronization in progress...*

All the process mentioned above was created because, in some rare cases, this problem could arise. And unfortunately, there is no other workaround except forcing a synchronization.

Thankfully, it only happens in some really rare cases, since the routine is set to an non-working hour time. But, as the process could take up to five seconds, it was necessary.

4.5.3 Copy to clipboard

One of the functionalities for an academic is to copy a link to a specific resource into his clipboard, so that he can publish this link on the Moodle platform. That way, students can directly access the resource by clicking on the link.

It offers some advantages compared to a simple updload of the PDF file on the Moodle platform: same process for internal and external resources, everything is centralized, a student can be directly linked to the sales website, etc.

But how to create a link that sends the user to the right resource, while taking its role into account ?

That is right. An academic that clicks on the link should have a different screen than a student or a manager.

This problem is addressed in two steps: creating the link, and sending the user to the corresponding role module.

Creating the link

This action is the first one that goes out of O.S. boundaries which means some *JavaScript* code will be required, since it is a client action, shown on Figure A-9.

It gets the resource identifier as parameter, `CurrentResourceId`, and a local variable that will contain the U.R.L. to copy, `ResourceDetailUrl`.

After some research, a post on the **Forum**[8] helped me understand how to copy something to the clipboard. The client action contains two *JavaScript* functions.

The first one, `GetEnvironmentUrl`, retrieves the environment of the application (thanks to an O.S. action), as the U.R.L. depends on it and changes if it is in development, testing or production, and returns it as `EnvironmentName` parameter:

```
$parameters.EnvironmentName = location.origin + "/"
```

Thanks to that, the complete U.R.L. has been formed in the `ResourceDetailUrl`, taking into acount the environment, the destination module (discussed below) and the resource identifier:

```
GetEnvironmentUrl.EnvironmentName +
```

[8][11]Ryan Rigby. *Copy to clipboard button.* url: `https://www.outsystems.com/forums/discussion/61018/copy-to-clipboard-button/#` (visited on 04/01/2023).

```
ResourceDetails/redirect?ResourceId=" +
CurrentResourceId
```

The second one, `CopyToClipboard`, copies in the user's clipboard the link to the corresponding resource:

```
navigator.clipboard.writeText(
    $parameters.Url
)
```

Then a success message is displayed and the link is copied.

Sending the user to the corresponding role module

To send the user to the correct module, taking its role into account, a redirect module has been created, *ReproRedirection*.

As already explained in sections 4.2.5 and 4.4.4, the module only contains one screen, reached at start, that redirects the user to another module depending on the user's role.

As we can see on Figure A-10, after the server action `CheckRole` that defines the role of the user, the client action, `RedirectToRightModule` redirects the user to the module that is related to its role.

Proceeding in this way allows a really strict control on who can access what, and a re-usability of already developed screens in other modules.

The redirection module is responsible for the role of the user and were it needs to be redirected and the screen of the destination module, `ResourceDetails`, is responsible for the validity of the accessed resource, the right to access it, and which type of resource it is.

The only improvement here would be to use something else than the identifier of the resource directly, which is an auto-incremented identifier. This problem is actually more general as it should be used throughout the entire application but will be tackled later.

4.5.4 Saving PDF files

Saving those files is, in itself, not a complicated functionality to implement in O.S..

Even though, it required a bit of reflection to make the system as safe as possible.

To prevent the user to upload fraudulent files, the extension is checked, the name is replaced, a maximum size is set up to the university standards.

After the security is set, the question comes to know how to upload them in the best way to prevent any issue for the user.

Files would be temporally uploaded in the database of the application and then, a routine will manage them to send them to a the real space of storage.

As already explained in the section 4.4.1, this last point was not implemented, but at least, all the procedure was already set up and defined.

4.5.5 Inconstancy of data used in Outsystems and the ESB

Considering the very new implementation of the environment for the university, a lot of adaptation in process and management needed to be created. Even more since the merger, everything leads to a lot of changes, and as workers are humans, sometimes errors are made during the process.

It is particularly the case for Integration Services that were developed to communicate with the E.S.B.. Some server actions were no longer correct or not even implemented. It became a big hindrance to development as, like milestones, it can not go further on specific features that requires those accesses to the E.S.B..

To prevent that to happen, all the required calls to the E.S.B. were already planned before the development phase but it was still not enough.

That is why some features of other modules have been developed when they were not supposed to be at first. Those were developed while waiting for the administration team to fix errors in Integration Services.

4.5.6 Adapting to Outsystems and its workflow

It may seem obvious, but one of the biggest challenges for this project was to develop a complex application in an environment that I had no prior knowledge of.

To make it working, I received a one week training. It was enough to introduce me to it, but not so as to become really proficient and efficient with it. That is why I had to regularly consult the **Documentation**[9] and the **Forum**.

After three months working on it, I could clearly see a big improvement.

[9][9]Outsystems. *Documentation*. url: `https://outsystems.com/documentation/` (visited on 02/27/2023).

Conclusion

This conclusion is separated into three parts as I have to conclude both on the application I worked on and on the internship I did, which was the very reason I had to work on this application and have written this bachelor thesis in the first place. And I wish to give my honest opinion about the first two ones.

The application

Its actual state

While the *ReproAcademic* module, even though close to completion, has not been done (the upload of PDF file is the only thing left), the work on the other modules that has been produced can definitely balanced for it.

Even more, a lot of features implemented in the *ReproAcademic* module will be easily refactored to be used in other modules such as the *ReproStudent* module that has quite the same workflow but with less features and applied to students instead of academics.

As it can be seen on the use-case diagram on Figure A-2, nearly a third of the features have been implemented.

All use cases for the *Academic* have been done except for the `Upload PDF file`.

Use cases `Display all resources`, `Sort resources`, `Filter resources`, `Search resources`, `Display all academics`, and `Search an academic` for the *Management* have been completed.

And the `Edit faculties colors` use case for the *Management administrator* has been done.

As of now, it can be estimated that the application should be done for the end of June, to be tested in the Summer, and be ready for September.

What still needs to be done

From the internship perspective, only the `Upload PDF file` use case still needs to be done.

As for the rest, it is mainly the *ReproStudent* and *ReproManagement* modules that has to be developed. With the advantage that development of the second has already started.

The internship

This part is way more subjective than the previous one as, instead of stating facts, I will give my personal opinion about the work that has been done, my experience, and will contain some of my thoughts about the project and related subjects.

What I gained from it

This project was actually extremely close to what I have been used to during my studies, whether it be at HeVinci School of Science and Technology or at Technological University Dublin (TUD) School of Computer Science. Even though, it brought me to use my knowledge to professional context and it was extremely rewarding and gave me a lot of confidence as I could clearly demonstrate all my potential and abilities, and see that I was not lacking in any aspect.

Being asked to continue working on the project as an employee is probably the biggest achievement I could get.

Moreover, I learned a lot about all the security measures and infrastructures used in order to make big companies, as a university with multiple campuses, to work properly.

What I have brought to the table

I could provide the university the ability to learn in a very fast pace and thus, be able to switch quickly to another development environment if needed.

And since I managed to master Outsystems (O.S.) very quickly, which also resulted in me working solely on this project, I could help my colleagues in their learning and understanding of it.

Furthermore, being extremely independent and structured in my work, and used to manage projects and client relations, allowed my project manager and my supervisor to focus on other tasks than monitoring my every activity, whether it be to interact with people from within and outside my department, setting up meetings and presentations, conducting the Terms of Reference, making an analysis, and developing the application.

Personal opinion

Used technologies

As for what I think about the technologies used to produce the application, I actually have mixed feelings. O.S. comes with good promises but does it really fulfill them ?

O.S. allows the user not to take care much neither about the structure of the produced application nor about the languages used to create it. It solely relies on the user using diagrams like process of coding. It this way, it allows less experimented users to develop applications as it becomes more abstract. It is also very useful for developers that do not want to take to much time on the front-end part of the application in such a way that, not only does it work like a framework such as bootstrap would, it also lightens the code development needed for it.

Furthermore, it has a lot of features that allow the user to build functions in a very fast way. The most important ones would be, using already existing templates, functions for login and register, or allowing quick requests to the database.

Working in a diagram-like environment seems promising for analysts as it has the same way of displaying process.

Nevertheless, not having the background of a computer scientist makes it hard to apprehend and understand and could easily lead to insecure code writing since it still heavily relies on the comprehension of advanced application structures of which the easiest to understand would be client-server pattern.

In addition, it requires computer scientist a lot of time to adapt to the workflow of O.S. since it is a completely new way to proceed and produce code. The reflection behind it is different too and producing good maintainable and readable code is harder than with conventional coding languages.

Moreover, information is not displayed at the same place everywhere and it can be a hustle to find it. And the AI helper that is omnipresent to help the user to find parameters more quickly or reformat instantly the code produced, is actually not performing that well. It often goes in the way of the user, most of the time, does not give useful parameters and warns the user on useless problems after its tests.

Besides, `commits` do not exist separately of pushing the code, which leads to having hours of work being done without any saves while refactoring the database because of unforeseen needed changes.

Finally, since it is proprietary, it brings with it all the inconveniences of being in a closed system: scarce documentation, a small community (even though it is reactive, it is not enough compared to what we can find for any coding languages), a place to share modules with others that has but few reviews and, for what I could see, poor quality of development, the impossibility to be future proof when retrieving the source code to work outside the environment.

In my opinion, only but a few will really take the risk to go and work within this environment that can reveal itself extremely frustrating to work with.

One last word

All of this leads me to say that I made a great work, and I am very happy that I could produce what was expected of me on time, which gives me this feeling of completion.

THIS PAGE INTENTIONALLY LEFT BLANK

Appendix A

Figures

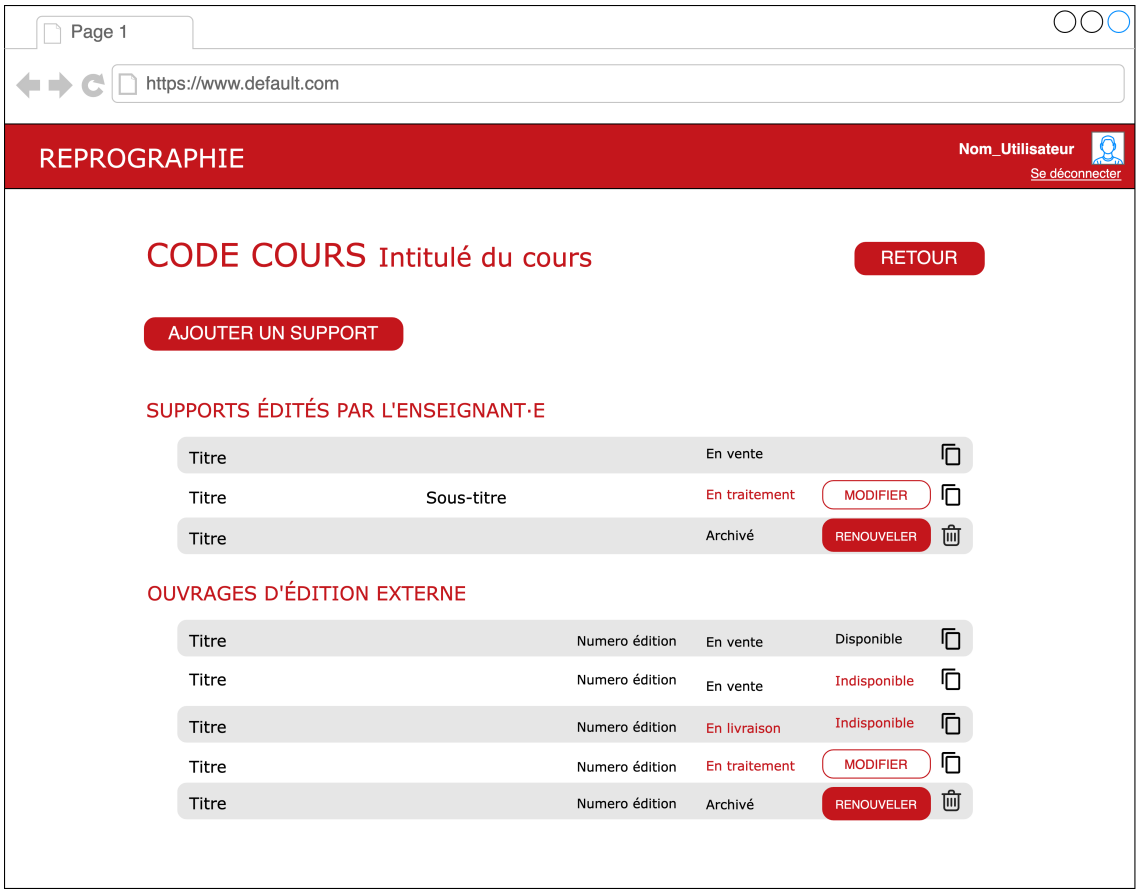

Figure A-1: Example of a wire-framed screen from the Terms of reference. This screen is part of the *ReproAcademic* module and lists all the resources of a learning unit of the current academic user.

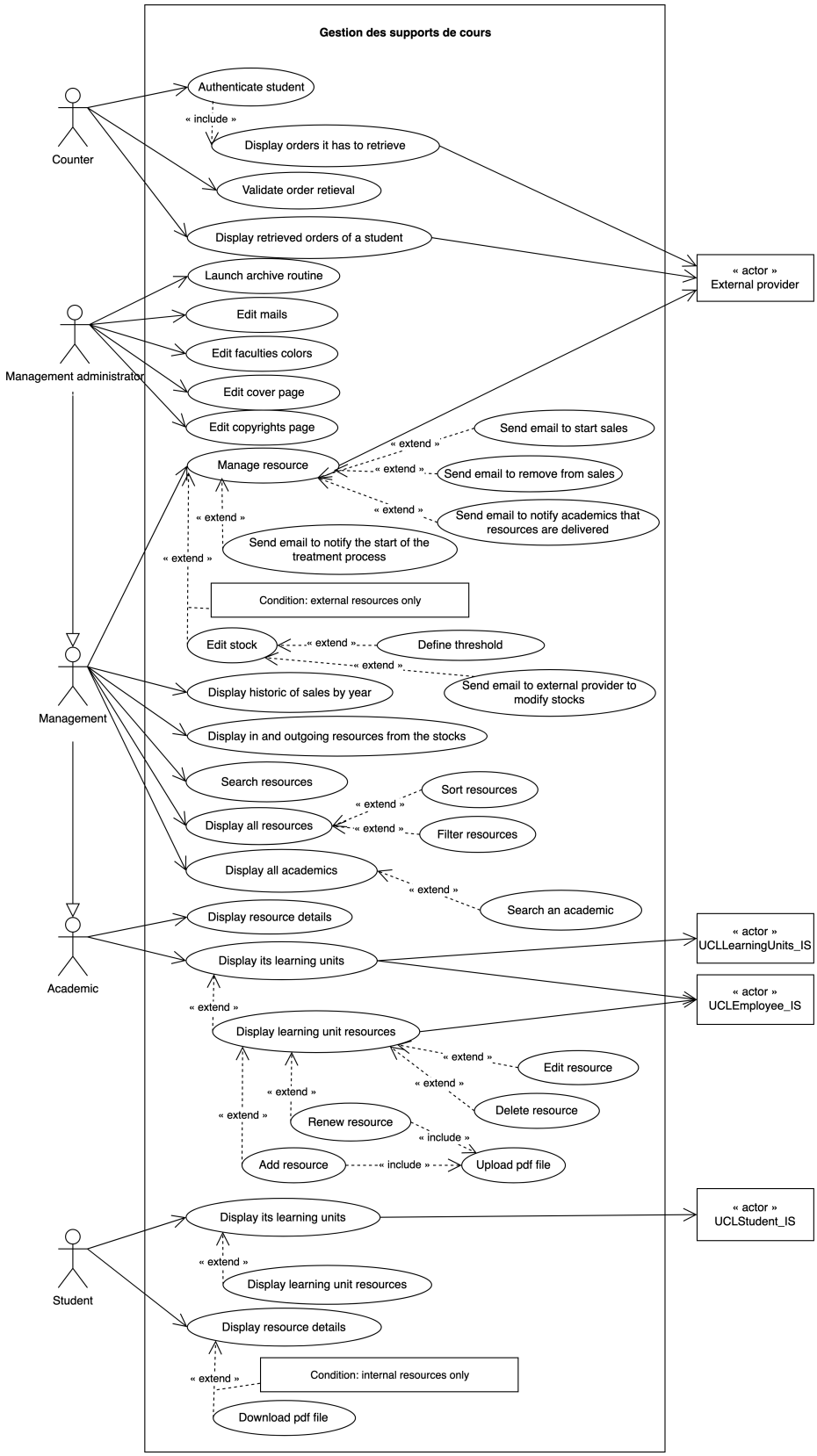

Figure A-2: Latest version of the use cases. Made for Outsystems

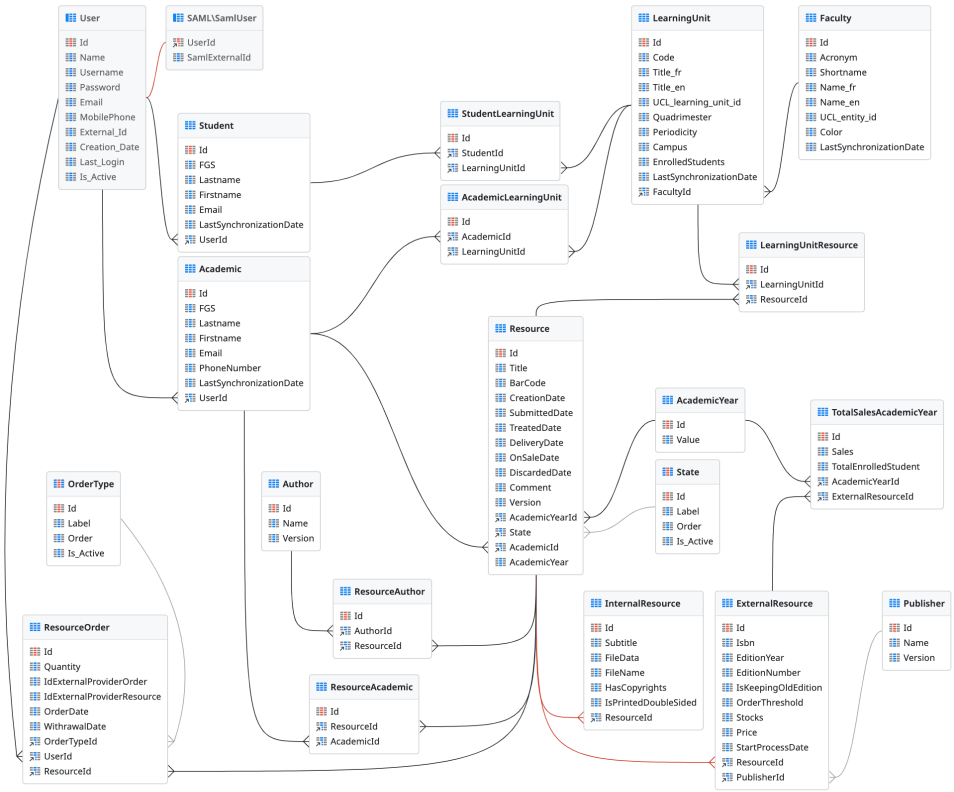

Figure A-3: Latest version of the entity diagram.

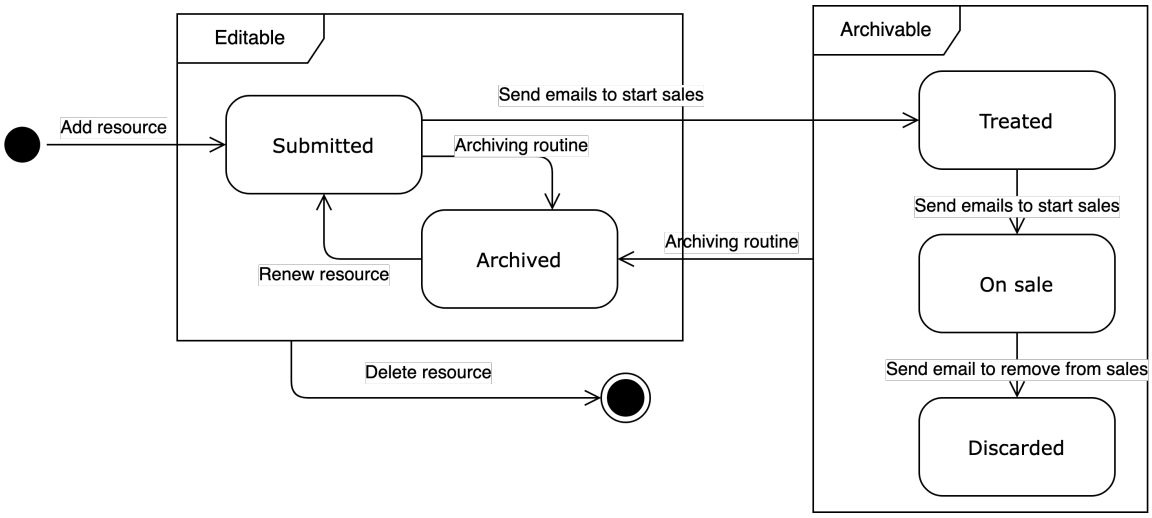

Figure A-4: State diagram for an internal resource.

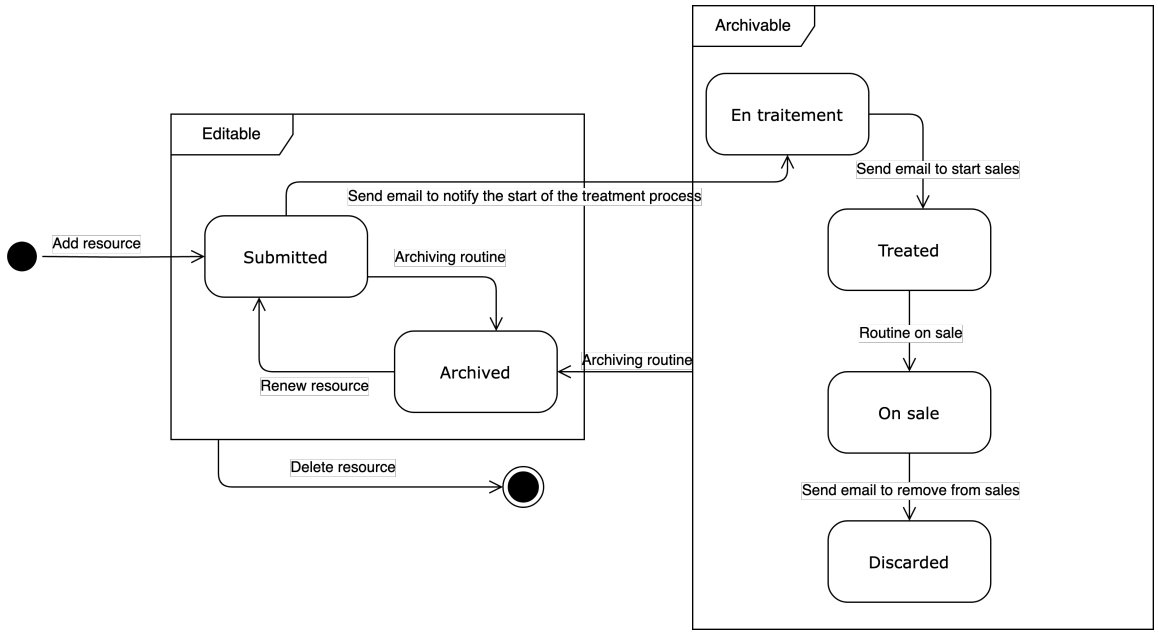

Figure A-5: State diagram for an external resource.

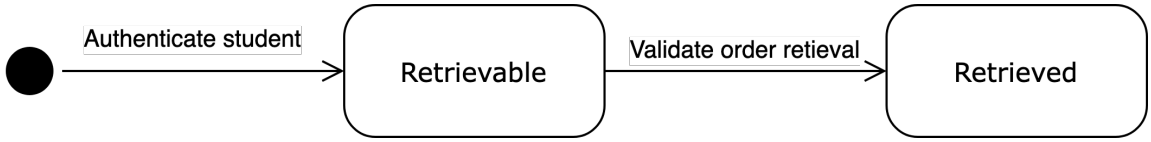

Figure A-6: State diagram for resource orders.

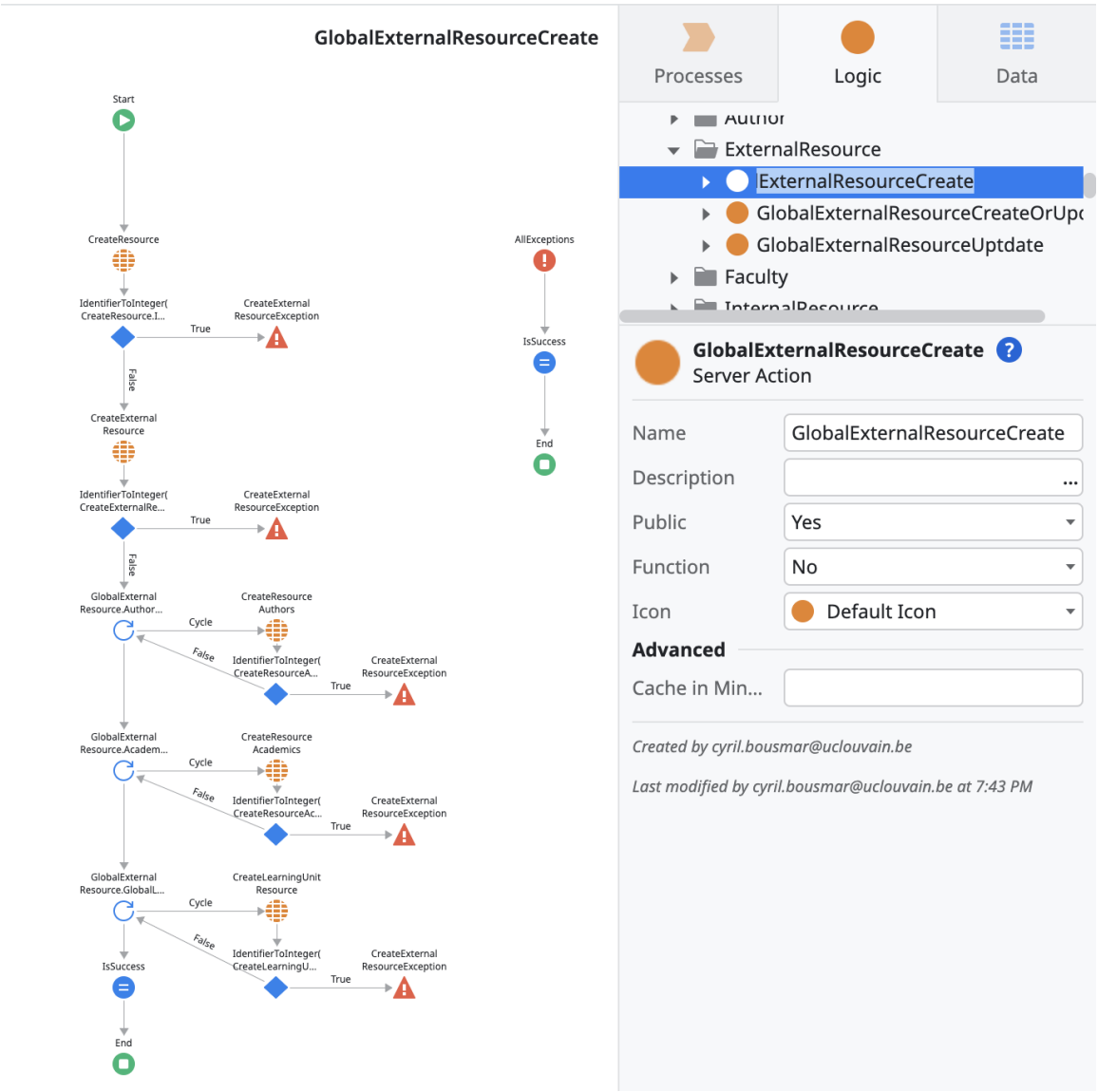

Figure A-7: Example of a server action in the *Reprographie_ CS* module to expose C.R.U.D. of the database to other modules.

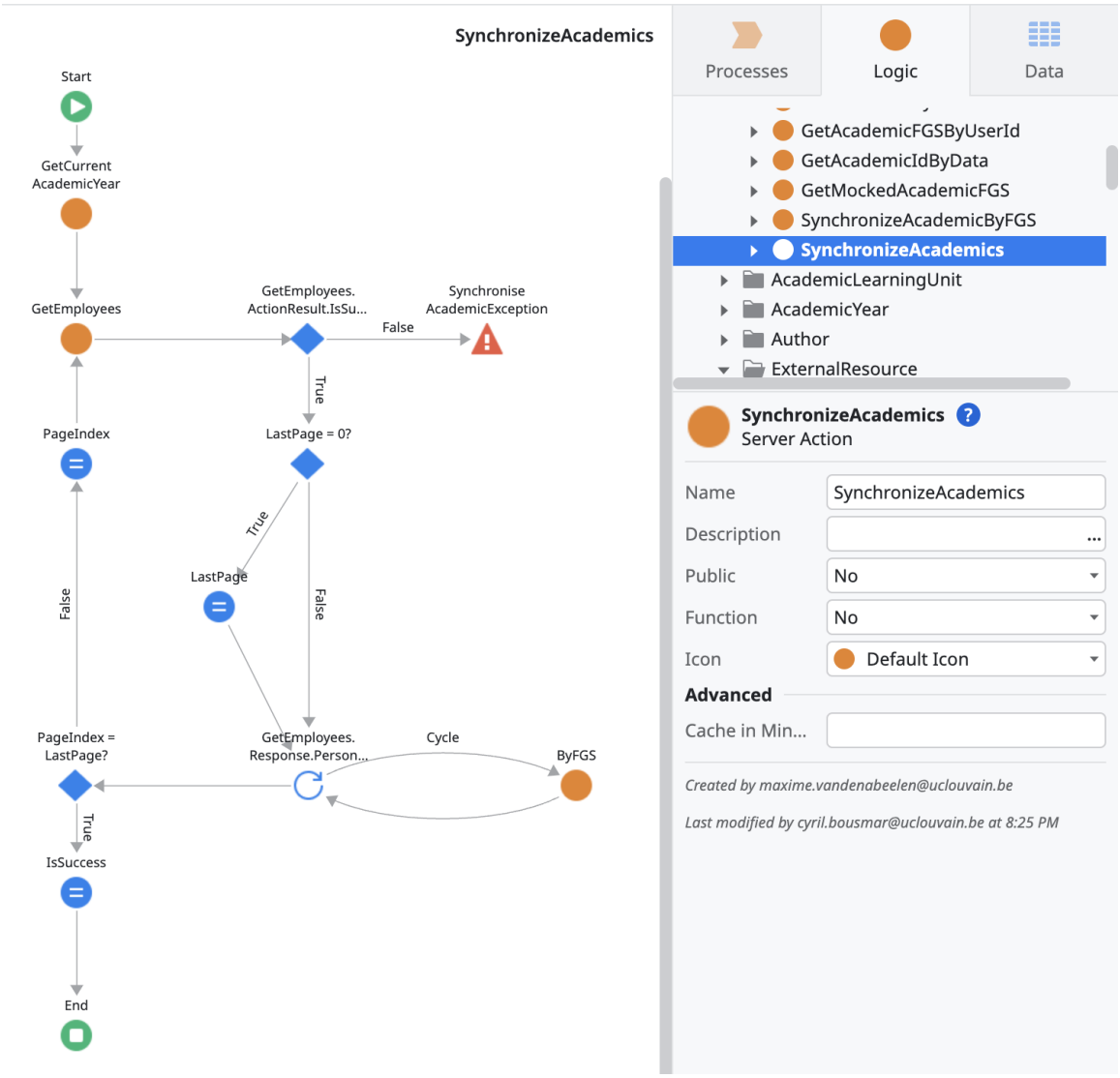

Figure A-8: Server action in the *Reprographie_CS* module to synchronize academic entity with all its records based on the data provided by the E.S.B..

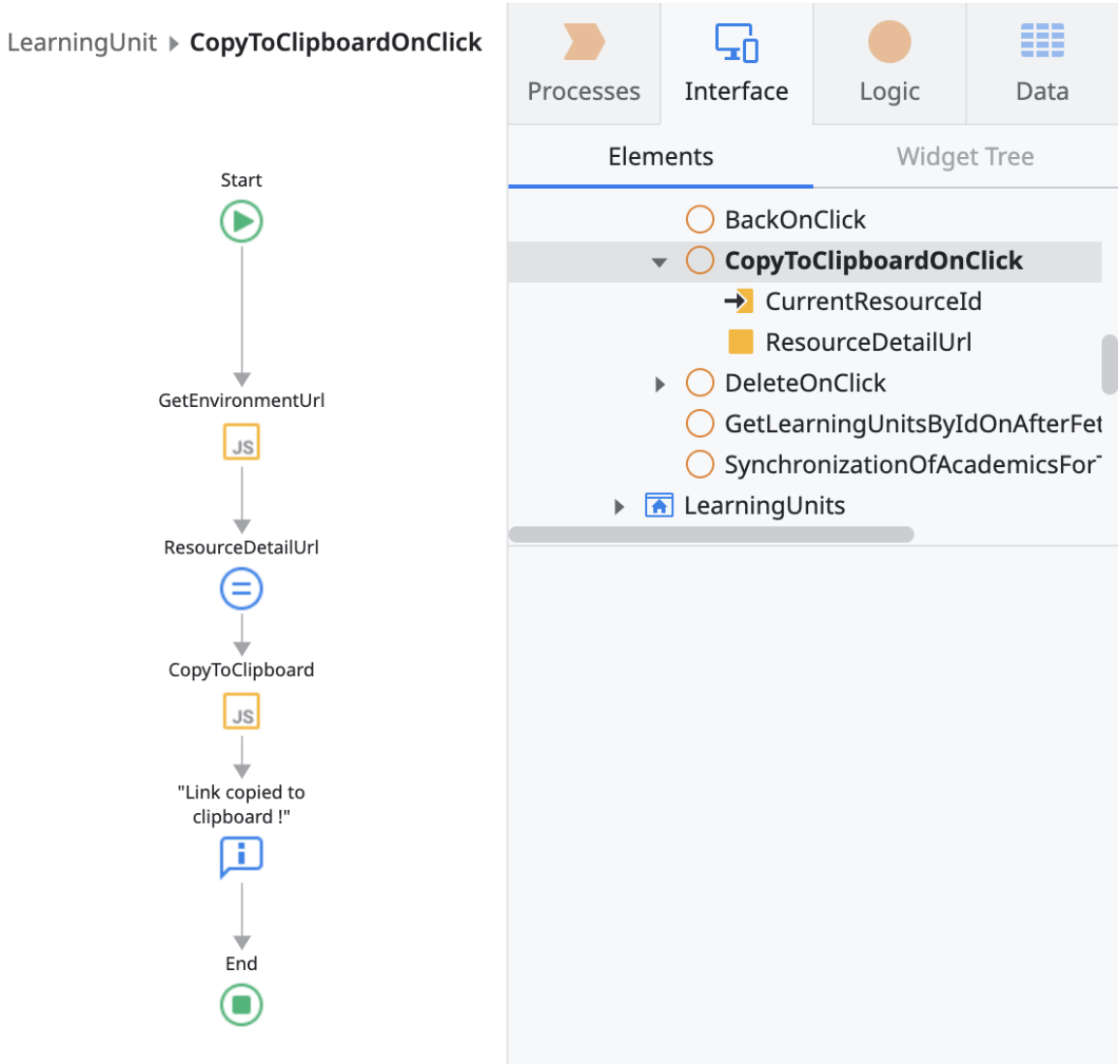

Figure A-9: Client action to copy to the user clipboard the link to a specific resource.

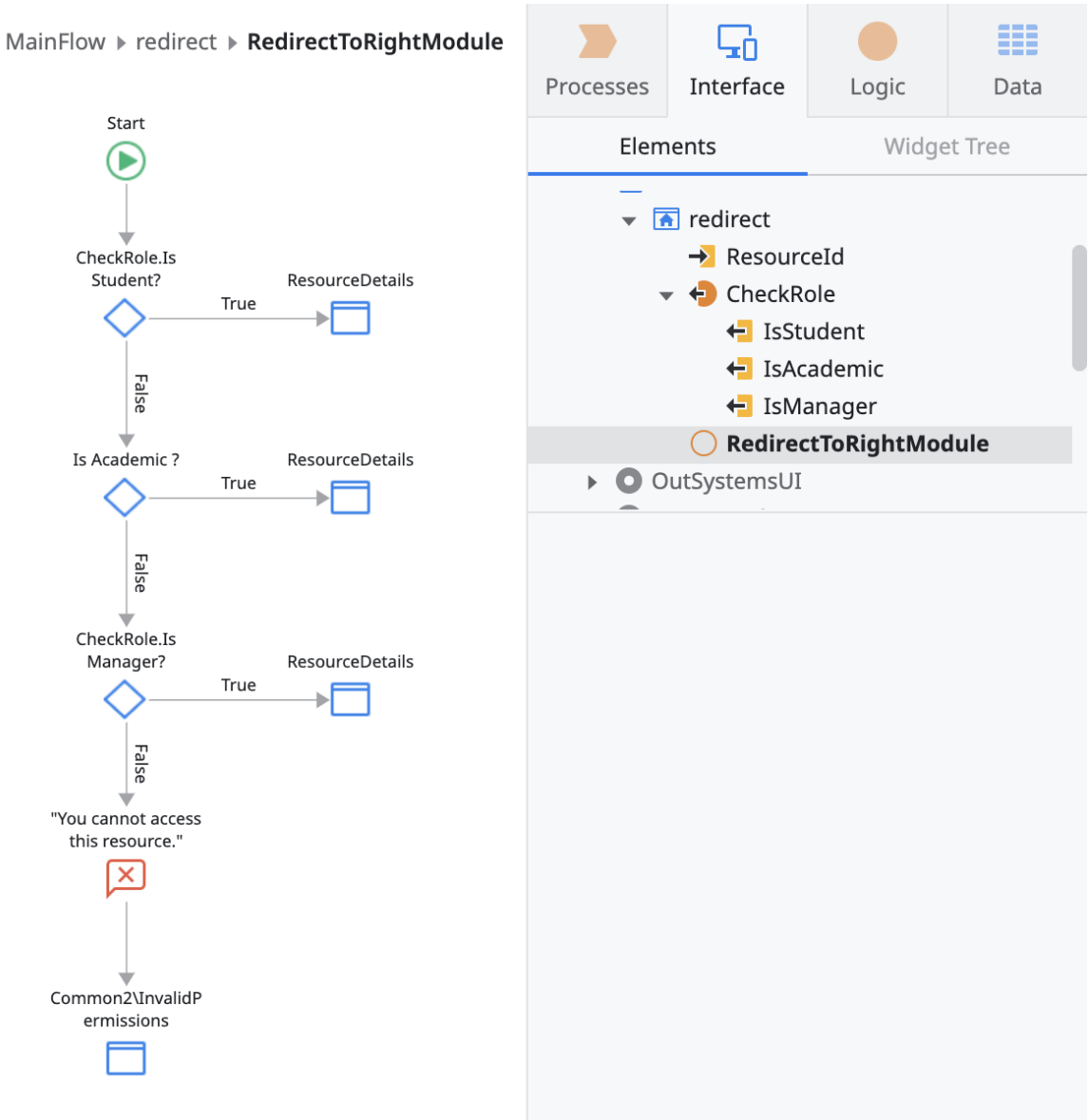

Figure A-10: Client action that redirects the user to a specific module depending on its role.

THIS PAGE INTENTIONALLY LEFT BLANK

Bibliography

[1] Cyril Bousmar. *Analyse technique - Application de gestion des supports de cours*. 1.2. Université Saint-Louis - Bruxelles. Brussels, Mar. 2023.

[2] Cyril Bousmar. *Cahier des charges - Application de gestion des supports de cours*. 1.2. Université Saint-Louis - Bruxelles. Brussels, Mar. 2023.

[3] Cyril Bousmar. *Technical analysis of LearningUnits*. Université Saint-Louis - Bruxelles. Brussels, Mar. 2023.

[4] Administration des relations extérieures et de la communication. *Historique*. Université catholique de Louvain. URL: `https://uclouvain.be/fr/decouvrir/historique.html` (visited on 03/31/2023).

[5] Administration des relations extérieures et de la communication. *Les missions et les objectifs*. Université catholique de Louvain. URL: `https://uclouvain.be/fr/decouvrir/missions-vision-valeurs.html` (visited on 03/31/2023).

[6] Service communication. *Les missions et les objectifs*. Université Saint-Louis - Bruxelles. URL: `https://www.usaintlouis.be/sl/1682.html` (visited on 03/31/2023).

[7] Service communication. *Une histoire ascensionnelle*. Université Saint-Louis - Bruxelles. URL: `https://www.usaintlouis.be/sl/3816.html` (visited on 03/31/2023).

[8] Leonardo Fernandes. *Ultimate PDF*. Outsystems Forge. URL: `https://www.outsystems.com/forge/component-overview/5641/ultimate-pdf` (visited on 04/12/2023).

[9] Outsystems. *Documentation*. URL: `https://outsystems.com/documentation/` (visited on 02/27/2023).

[10] Outsystems. *What is Outsystems ?* URL: `https://www.outsystems.com/` (visited on 02/27/2023).

[11] Ryan Rigby. *Copy to clipboard button*. URL: `https://www.outsystems.com/forums/discussion/61018/copy-to-clipboard-button/#` (visited on 04/01/2023).

Acronyms

.NET Dotnet. 22

A.P.I. Application Programming Interface. 29, 37, 39

Azure AD Azure Active Directory. 14, 29

C.R.U.D. Create, Read, Update, Delete. 5, 25, 37, 43, 62

C.S. Core Service. 37

E.S.B. Enterprise Service Bus. 5, 14, 15, 29, 39, 41, 49, 63

F.G.S. Fichier Général du Signalétique (General Signage File). 31, 39, 41, 45

HeVinci Haute École Léonard de VINCI. 52

I.S. Integration Service. 37, 39, 49

JS JavaScript. 22

N.O.M.A. Numéro de matricule (Student Number). 38

O.S. Outsystems. 8, 9, 14, 15, 21, 23, 30–32, 34–37, 41, 44, 45, 47, 49, 53, 54

Office of course materials Bureau des supports de cours. 8, 17, 19, 33, 34, 38

OSForge Outsystems Forge. 22, 36

S.A.A.S. Software as a Service. 21

S.S.O. Single sign-on. 14, 15, 34

Shibboleth Shibboleth Consortium. 14

SQL Structured Query Language. 22

TUD Technological University Dublin. 52

U.I. User Interface. 24

U.M.L. Unified Modeling Language. 30, 31, 33

U.R.L. Uniform Resource Locators. 36, 38, 47

UCLouvain Université catholique de Louvain. 3, 7–9, 12, 14, 15, 21, 28, 29, 32, 34

UCLouvain-SLB UCLouvain Saint-Louis - Bruxelles. 5, 11, 28

USLB Universié Saint-Louis - Bruxelles. 3, 7, 9, 11–13, 28

V.M. Virtual Machine. 8

X.M.L. Extensible Markup Language. 22